

מזהה סטודנט

תאריך בחינה

מזהה קורס

שם קורס

מרצה

יום שני, 21 ביולי 2025

20905

שפות תכנות

דני כלפון

ניקוד שאלות פתוחות	ציון מבחן מקורי	ציון מבחן סופי
100.00	100.00	100.00

סיכום

מספר שאלה	ניקוד	ניקוד מירבי
1	35.00	35.00
2	40.00	40.00
3	25.00	25.00

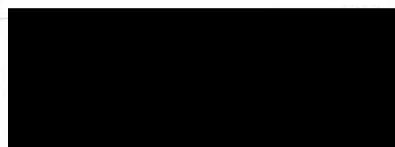
19

מועד

50902

מספר הקורס

מספר תעודת זהות (9 ספרות)



לשימוש הבודק

הפירוץ לפי הסדר הבא:

1. שאלה 3

2. שאלה 1

3. שאלה 2

שאלה 3
בניית סוגי הנתונים והמשתנים

Expressions	Type variables
1 int	t_0
2 $\text{proc}(a:\text{?}) (a) \text{ } \square$	t_1
3 $(a) \text{ } \square$	t_2
4 $(p \text{ proc}(b:\text{int}) \text{ zero?}(b))$	t_3
5 $\text{proc}(b:\text{int}) \text{ zero?}(b)$	t_4
6 $\text{zero?}(b)$	t_5
p	t_p
a	t_a
b	t_b



הערה: סדר הנתונים והמשתנים חייב להיות זהה לסדר שבו הם מופיעים בפרוטוקול.

כך נחבר משוואות סוגיות והמשתנים שנקבעו

Expressions	Equations
1	$t_0 = t_3$ $t_p = t_1$
2	$t_1 = t_a \rightarrow t_2$
3	$t_a = \text{int} \rightarrow t_2$
4	$t_p = t_4 \rightarrow t_3$
5	$t_4 = t_b \rightarrow t_5$ $t_b = \text{int}$



Expressions	Equations
[6]	$t_s = \text{bool}$ $t_b = \text{int}$



ג. המצב החדש של האלקטרוניקה הוא $t_b = \text{int}$ (כבר, נכון)

Equations	Substitutions
$t_0 = t_3$ $t_p = t_1$ $t_1 = t_a \rightarrow t_2$ $t_a = \text{int} \rightarrow t_2$ $t_p = t_4 \rightarrow t_3$ $t_4 = t_b \rightarrow t_5$ $(*) t_b = \text{int}$ $(*) t_s = \text{bool}$ 	

לדוגמה, המערכת מקבלת טיפוסים $(*)$ ללא יציאה.

Equations	Substitutions
$t_0 = t_3$ $t_p = t_1$ $t_1 = t_a \rightarrow t_2$ $t_a = \text{int} \rightarrow t_2$ $t_p = t_4 \rightarrow t_3$ $(*) t_4 = \text{int} \rightarrow \text{bool}$	$t_b = \text{int}$ $t_s = \text{bool}$

לשם: * (לשם: יו"ן) (קב)

לשימוש הבודק

Equations	Substitutions
$t_0 = t_3$	$t_b = \text{int}$
$t_p = t_1$	$t_s = \text{bool}$
$t_1 = t_a \rightarrow t_2$	$t_4 = (\text{int} \rightarrow \text{bool})$
$t_a = \text{int} \rightarrow t_2$	
(A) $t_p = t_4 \rightarrow t_3$ ($\text{int} \rightarrow \text{bool}$)	

לשם: * (לשם: יו"ן) (קב)

Equations	Substitutions
$t_0 = t_3$	$t_b = \text{int}$
(*) $t_p = t_1$ ($\text{int} \rightarrow \text{bool}$) $\rightarrow t_3$	$t_s = \text{bool}$
$t_1 = t_a \rightarrow t_2$	$t_4 = (\text{int} \rightarrow \text{bool})$
$t_a = \text{int} \rightarrow t_2$	$t_p = (\text{int} \rightarrow \text{bool}) \rightarrow t_3$

$$t_1 = t_4 \rightarrow t_2$$

$$t_2 = t_3$$

$$(*) t_a = (\text{int} \rightarrow \text{bool})$$

לשם: * (לשם: יו"ן) (קב)

$$t_1 = (\text{int} \rightarrow \text{bool}) \rightarrow t_3$$

$$t_a = (\text{int} \rightarrow \text{bool})$$

Equations	Substitutions
$t_0 = t_3$	$t_b = \text{int}$
t_0 $t_2 = t_3$	$t_s = \text{bool}$
$t_1 = t_a \rightarrow t_2$ ($\text{int} \rightarrow \text{bool}$)	$t_4 = (\text{int} \rightarrow \text{bool})$
$t_4 = \text{int} \rightarrow t_2$	$t_p = (\text{int} \rightarrow \text{bool}) \rightarrow t_3$
	$t_1 = (\text{int} \rightarrow \text{bool}) \rightarrow t_3$
	$t_a = (\text{int} \rightarrow \text{bool})$

[illegible]

Equations	Subst.
$t_0 = t_3$	$t_6 = \text{int}$
$(*) \text{ bool } t_2 = t_3$	$t_5 = \text{bool}$
$(\kappa) t_1 = (\text{int} \rightarrow \text{bool}) \rightarrow t_2$	$t_4 = (\text{int} \rightarrow \text{bool})$
then	$t_7 = (\text{int} \rightarrow \text{bool}) \rightarrow t_3$
	$t_1 = (\text{int} \rightarrow \text{bool}) \rightarrow t_3$
	$t_a = (\text{int} \rightarrow \text{bool})$
	$t_2 = \text{bool}$

רמז (א) (שני משולבים) ערך מ' . נ' (ובתו) (מחיר)

Equations	subst.
$t_0 = t/3$ $bool$	$t_b = int$ $t_s = bool$ $t_n = (int \rightarrow bool)$ $t_p = (int \rightarrow bool) \rightarrow bool$ $t_1 = (int \rightarrow bool) \rightarrow bool$ $t_a = (int \rightarrow bool)$ $t_2 = bool$
	$t_3 = bool$ $t_4 = (int \rightarrow bool)$

לעזר אחר $t_0 = \text{bool}$ לצד ימין וסמל.

מאחר ו- t_0 הוא משתנה שמייצג את טיפוס t_0 הביא קבץ שהטיפוס ה"ל" הוא bool. ✓

(המשק המבחן במחזור הבא)

טבלה:

~~type-val סמל חזק~~
~~type-str string?~~
~~(type-val "bool")~~
~~get-val-type: exprval $\rightarrow$ string~~

טבלה:

~~(int/bool/float) type~~
~~procedure~~
~~proc-val~~
~~proc-exp~~
~~apply-procedure~~
~~do-overload:~~
~~overloaded $\times$ type $\times$ var $\times$ body $\times$ environment~~
 ~~$\rightarrow$ overloaded~~

אשר

the-grammar- δ $\rho \cdot 10 \cdot 2 \cdot 1 \cdot 2$

(expression

(expression

(expression



("isProc?" "(" expression ")")

[illegible]

(--)
Num/bool/proc-val



type-str? היתר א אצו

024733

(define type-str?

(lambda (st)

(and (string? st) (or (equal? st "num")

(equal? st "bool")

(equal? st "type")(equal? st "proc"))

)))

expval $\rightarrow$ type ϕ (המח) אר נגזר

(define expval $\rightarrow$ type

(lambda (val)

(cases expval val

(type-val (st) st)

(else (report-expval-extractor-error 'type val))))))

כס נוסף אר אכן הפירוט של הביטויים החדשים
value-of $\rightarrow$ מח

(define value-of

(lambda (exp env)

(cases expression exp

(...) $\leftarrow$ $\begin{matrix} \text{מימין פירוט} \\ \text{הביטויים הקיימים} \\ \text{proc-} \end{matrix}$

(get-type-exp (id)

(let ((val1 (apply-env env id))))

(type-val (get-val-type val1))

))

;; המשך במה הבא

(isBool?-exp (ex)

(let ((type-st (expval→type (value-of ex env))))

(bool-val (equal? type-st "bool"))

))

(isNum?-exp (ex)

(let ((type-st (expval→type (value-of ex env))))

(bool-val (equal? type-st "num"))

))

(isProc?-exp (ex)

(let ((type-st (expval→type (value-of ex env))))

(bool-val (equal? type-st "proc"))

))

)))

get-val-type נחשב רק למחלקת הנתונים

(define get-val-type

(lambda (val)

(cases expval val

(num-val (n) "num")

(bool-val (b) "bool")

(proc-val (p) "proc")

(type-val (t) "type")

)))

ציון שאלה 1:

עדכון דקדוק השפה

עדכון דקדוק ביטוי 3/3 - get-type-exp

עדכון דקדוק ביטויים 6/6 - isBool, isNum, isProc

עדכון expval עם ייצוג תשובה לתיאור טיפוס - 6/6

מימוש התנהגות:

מימוש התנהגות ביטוי 5/5 - get-type-exp

מימוש התנהגות ביטויים 15/15 - isBool, isNum, isProc

שאלה 2

האם, לעדכן את כל המידע של proc-exp ושל כל המידע של overloaded-exp ושל type ושל the-grammar.

(expression

("proc" "(" type identifier ")" expression)
proc-exp)

(expression

("overload" identifier "with" "(" type identifier ")" expression)
overload-exp)

(type ("int") int-type)

(type ("bool") bool-type)

(type ("func") func-type)

~~האם, לעדכן את כל המידע של proc-exp ושל overloaded-exp ושל type ושל the-grammar.~~

overloaded המידע של

(define-datatype overloaded overloaded?

(overloaded-proc

(int-proc ~~proc-exp~~ proc?)

(bool-proc ~~proc-exp~~ proc?)

(func-proc ~~proc-exp~~ proc?)

))

024733

expval & proc-val 'k) → k | 38)

(define-datatype expval expval?)

$$(\dots) \leftarrow \begin{matrix} \text{מחזורי} & \text{מחזורי} \\ \text{פולינומי} & \text{מחזורי} \end{matrix}$$

(proc-val (proc ~~overloaded?~~ overloaded?)))

)

נוסח למקרה הנתונים

7

לכן "הוא" ~~procedural~~

(define-data-type proc proc?)

(procedure

(var identifier?)

(body expression?)

(save-env environment?)

~~Handwritten signature~~

empty-proc

)

תחת Value-of, נסבין את אופן הפירוש של proc-exp ופולין את אופן הפירוש של overload-exp

- קצת 'מזל פז' -

THE UNIVERSITY OF CHICAGO

בס

1. כלל הנחיות
 2. כלל הנחיות
 3. כלל הנחיות

בס

בס

בס

בס

בס

בס

בס

בס

בס

בס

בס

בס

בס

בס

בס

בס

בס

בס

בס

```

(overload-exp (p-name typ var body)
  (let* ((p-ref (apply-env env env p-name))
        (p-val (deref p-ref))
        (p-ov (expval→proc p-val)))
    (setref! p-ref (proc-val (do-overload
                               p-ov typ var body env))))
  ))
)))

```

apply-procedure ↗ | 38

```

(define apply-procedure
  (lambda (proc1 val)
    (cases overloaded proc1
      (overloaded-proc (int-proc bool-proc func-proc)
        (cases expval val
          (num-val (v)
            (cases proc int-proc
              (empty-proc ()) (exp!error "No number proc found!"))
              (procedure (var body saved-env)
                (value-of body (extend-env var (newref val)
                                             saved-env))))
            ))
          (bool-val (v)
            (cases proc bool-proc
              (empty-proc ()) (exp!error "No bool proc found!"))
              (procedure (var body saved-env)
                (value-of body (extend-env var (newref val)
                                             saved-env))))
            ))
          )))
    ))

```


(proc-val (v))

✓ (cases proc func-proc
(empty-proc () (eopl:error
"no func proc found!"))

(procedure (var ~~body~~ body saved-env)
(value-of body
(extend-env var (newref val)
saved-env)))

))

(else (eopl:error "Unsupported type found!"))

)))

))

do-overload 1587 1310 11 1111 11 11

(define do-overload

(lambda (old-ov typ var body env)

(cases overloaded old-ov

(overloaded-proc (int-proc bool-proc func-proc)

(cases type typ

(int-type () (overloaded-proc

(procedure var body env)

bool-proc
func-proc))

(bool-type () (overloaded-proc

int-proc

(procedure var body env)

func-proc))

1 11 1111 11

(func-type () (overloaded-proc
int-proc

bool-proc

(procedure var body env)))



)))

))

ציון שאלה 2:

עדכון דקדוק השפה

עדכון דקדוק ביטוי 2/2 - proc-exp

עדכון דקדוק ביטוי 4/4 - overload-exp

עדכון דקדוק של 3/3 - type

עדכון הייצוג של פרוצדורה - 6/6

מימוש התנהגות:

עדכון התנהגות ביטוי 5/5 - proc-exp

מימוש התנהגות ביטוי 10/10 - overload-exp

עדכון התנהגות של 10/10 - call-exp+applyprocedure

